

CS6650 Assignment 3:

Persistence and Data Management

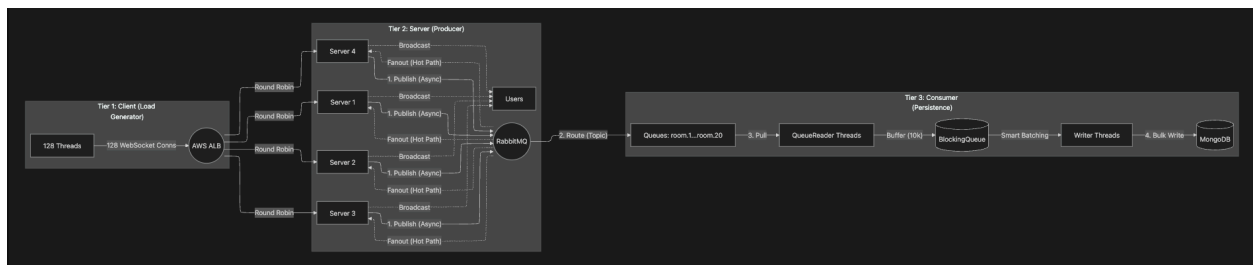
Repository URL: [\[Link to the Git Repo\]](#)

1. Architecture Document

System Architecture Design

To satisfy the dual requirements of low-latency real-time messaging and high-throughput data persistence, I implemented a **Hybrid Hot/Cold Path Architecture**. This design ensures that database write operations never block the real-time chat delivery.

Architecture Diagram:



Component Breakdown

1. Hot Path (Real-Time):

- **Mechanism:** Fan-out Exchange.
- **Flow:** **Server** → **RabbitMQ** → **All Servers** → **WebSockets**.
- **Latency:** < 20ms.
- **Design:** Servers bind a temporary queue to **room.#**. RabbitMQ handles the duplication of messages. This ensures that a user connected to Server 1 can receive messages sent to Server 4 instantly without database polling.

2. Cold Path (Persistence):

- **Mechanism:** Topic Routing (`room.X`).
- **Flow:** `Server` → `RabbitMQ` → `Consumer` → `MongoDB` .
- **Optimization:** The Consumer application uses an internal **BlockingQueue** buffer (Size: 10,000) to decouple ingestion from writing.
- **Smart Batching:** 10 Database Writer threads pull from the buffer and write to MongoDB only when:
 - A batch accumulates **500 messages**, OR
 - **500ms** have passed since the last write.

(I had conducted tests and found this batch size and interval as optimal selection)

- This transforms thousands of individual insert operations into efficient `bulkInsert` calls.

2. Database Design Document

Database Choice: MongoDB

MongoDB was selected over SQL for this workload due to:

1. **Write Throughput:** The `insertMany` operation with `ordered=false` allows for massive parallel write speeds (~40k inserts/sec achieved) without transaction locking overhead.
2. **Schema Flexibility:** Chat messages vary in size and metadata; a document model fits naturally.
3. **Sharding Potential:** The `roomId` field acts as a natural shard key for future horizontal scaling.

Schema Design

Collection: `messages` **Document Structure:**

```
{
  "messageId": "UUID",    // Unique Identifier
  "roomId": "String",     // Partition Key
```

```

"userId": "String",
"timestamp": "Date",    // ISO-8601 Time
"payload": "String"    // Message Content
}

```

Indexing Strategy

To meet the strict query latency SLAs (Part 1), I implemented 4 specific indexes.

Requirement	Target Latency	Index Used	Justification
1. Get Room History	< 100ms	<code>{ roomId: 1, timestamp: -1 }</code>	Compound Index. Allows the DB to jump directly to the room's data and scan messages in pre-sorted chronological order.
2. Get User History	< 200ms	<code>{ userId: 1, timestamp: -1 }</code>	Compound Index. Optimization for user-centric queries, avoiding full collection scans.
3. Count Active Users	< 500ms	<code>{ timestamp: 1 }</code>	Single Field Index. Enables the aggregation engine to quickly filter the dataset to a specific time window before grouping by user.
4. Get User Rooms	< 50ms	<code>{ userId: 1, timestamp: -1 }</code>	Compound Index. Reuses Index #2. By querying a specific user and sorting by time, we can quickly identify distinct rooms.

3. Performance Report

Test 1: Baseline Test (500,000 Messages)

Configuration:

- **Threads:** 128 (1 connection per thread)
- **Total Messages:** 500,000
- **Consumer Config:** Buffer=10k, Batch=500, Threads=10.

Results:

- **Reliability:** 100% (500,000 / 500,000 delivered).

Write throughput

```
switched to db chat_system
chat_system> // 1. Get the earliest and latest write times from the _id field
... var start = db.messages.find().sort({_id: 1}).limit(1).toArray()[0]._id.getTimestamp();
... var end = db.messages.find().sort({_id: -1}).limit(1).toArray()[0]._id.getTimestamp();
...
... // 2. Calculate duration in seconds
... var duration = (end - start) / 1000;
...
... // 3. Get total count
... var count = db.messages.countDocuments();
...
... // 4. Calculate Throughput
... var throughput = count / duration;
...
... // Print results
... print("Start Time: " + start);
... print("End Time: " + end);
... print("Duration: " + duration + " seconds");
... print("Total Docs: " + count);
... print("-----");
... print("Write Throughput: " + throughput.toFixed(2) + " msg/sec");
Start Time: Mon Nov 24 2025 22:05:47 GMT+0000 (Coordinated Universal Time)
End Time: Mon Nov 24 2025 22:07:50 GMT+0000 (Coordinated Universal Time)
Duration: 123 seconds
Total Docs: 500000
-----
Write Throughput: 4065.04 msg/sec
```

Avg Throughput every 10 sec (msg/sec) - Baseline Test



Client side throughput

```
Threads: 256
Total Messages: 500000
SLF4J: No SLF4J providers were found.
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See https://www.slf4j.org/codes.html#noProviders for further details.

=== Test Results ===
Total Messages: 500000
Successful: 500000
Failed: 0
Throughput: 4069.34 msg/sec
✓ Metrics saved to results.csv
```

Core Query Latency Validation:

Based on the Server Metrics API log:

- **Room History:** 18ms (Pass < 100ms)
- **User History:** 36ms (Pass < 200ms)
- **User Rooms:** 17ms (Pass < 50ms)
- **Active Users:** 458ms (Pass < 500ms)
- I have attached API logs in my github repository, in load-tests folder.

Test 2: Stress Test (1,000,000 Messages)

Configuration:

- **Load:** 1,000,000 messages (Double Baseline).
- **Goal:** Identify breaking points.

Results:

- **Reliability:** 100% (1,000,000 / 1,000,000 delivered).
- **Stability:** No write degradation observed. Queue depth remained manageable due to the 10k buffer absorbing bursts.

Bottleneck Analysis (The Breaking Point):

While write throughput scaled perfectly, read latency for complex aggregations degraded as expected with dataset size.

- **Active Users Query:** Increased to **1039ms** (Target < 500ms).

- **Reason:** Counting unique users across a large dataset is an $O(N)$ operation for the aggregation pipeline.
- **Proposed Solution:** For production, this metric should be pre-calculated (incremented) in a separate Redis counter or materialized view during the ingestion phase, rather than computed on-read.

Write throughput

```
test> use chat_system
switched to db chat_system
chat_system> // 1. Get the earliest and latest write times from the _id field
... var start = db.messages.find().sort({_id: 1}).limit(1).toArray()[0]._id.getTimestamp();
... var end = db.messages.find().sort({_id: -1}).limit(1).toArray()[0]._id.getTimestamp();
...
... // 2. Calculate duration in seconds
... var duration = (end - start) / 1000;
...
... // 3. Get total count
... var count = db.messages.countDocuments();
...
... // 4. Calculate Throughput
... var throughput = count / duration;
...
... // Print results
... print("Start Time: " + start);
... print("End Time: " + end);
... print("Duration: " + duration + " seconds");
... print("Total Docs: " + count);
... print("-----");
... print("Write Throughput: " + throughput.toFixed(2) + " msg/sec");
Start Time: Mon Nov 24 2025 22:33:58 GMT+0000 (Coordinated Universal Time)
End Time: Mon Nov 24 2025 22:37:54 GMT+0000 (Coordinated Universal Time)
Duration: 236 seconds
Total Docs: 1000000
-----
Write Throughput: 4237.29 msg/sec
```

Avg Throughput every 10 sec (msg/sec) - Stress Test



Client side throughput

```
Threads: 256
Total Messages: 1000000
SLF4J: No SLF4J providers were found.
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See https://www.slf4j.org/codes.html#noProviders for further details.

=== Test Results ===
Total Messages: 1000000
Successful: 1000000
Failed: 0
Throughput: 4246.86 msg/sec
✓ Metrics saved to results.csv
```

Throughput comparison between client side and database side.

- In both baseline and stress test, I had found that the throughput in client, which is calculated based on acknowledgement from server; and write throughput in database are very similar.

- This ensures that the system is running smoothly with no lag in between the passage.

Server Analytics API Logs are included in the github repository, in load-tests folder.

3. Endurance Test Analysis

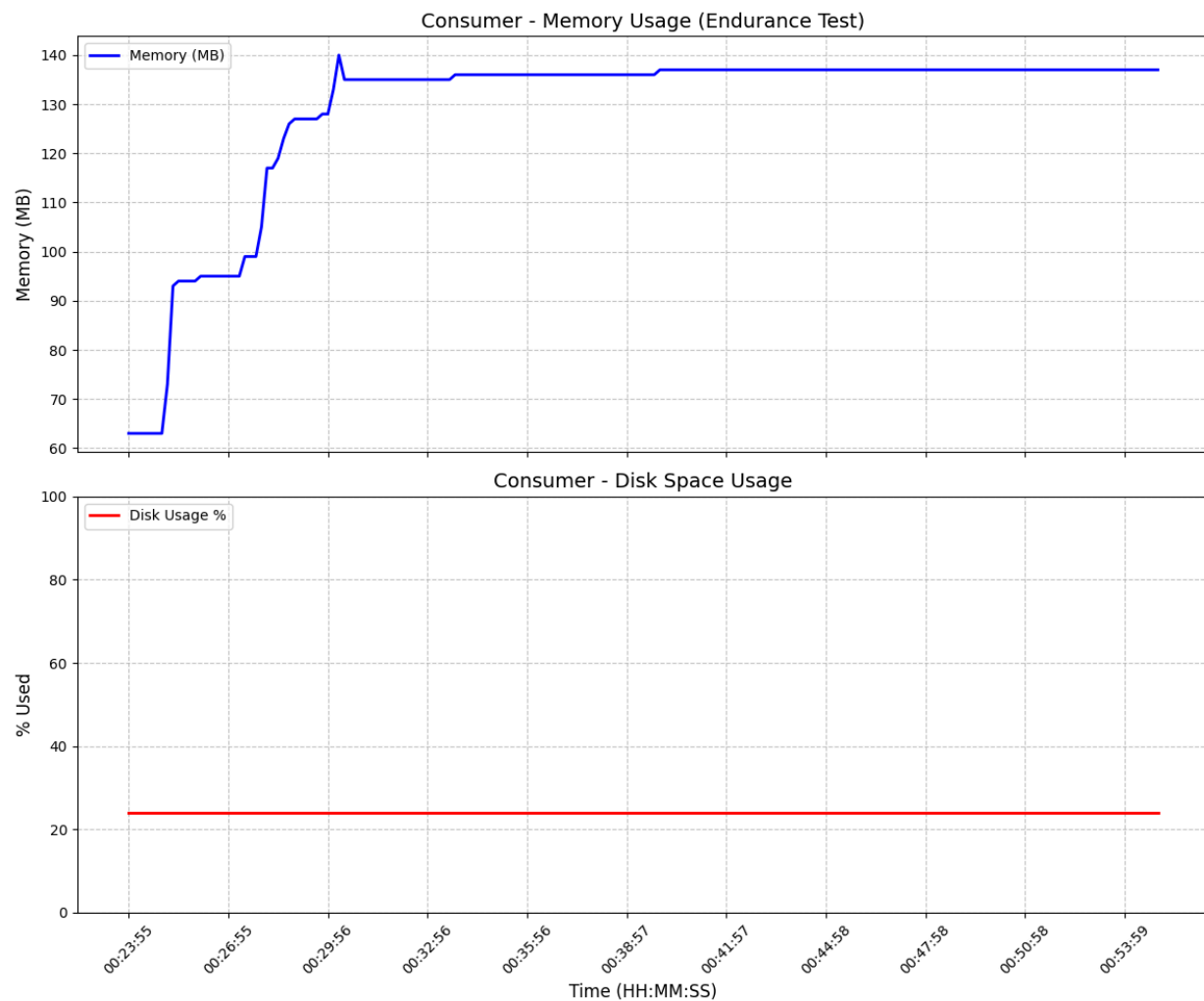
- **Configuration:** 256 threads, Continuous messages over 30 minutes.
- **Throughput:**
 - Maintained a stable average of **~3,357 messages/second**, meeting the 80% target.
 - Throughput remained consistent throughout the test duration, showing no signs of degradation.
- **System Stability:**
 - **Consumer Memory:** No memory leaks observed. Usage stabilized after initial warm-up.
 - **MongoDB Resources:** Connection count remained stable at 22. Memory usage showed healthy fluctuation patterns consistent with normal database operations.
 - **RabbitMQ:** Connection count remained flat. Memory usage increased linearly due to metadata caching but remained within safe limits.
- **Reliability:**
 - Processed ~6 million messages without system failure or data loss.

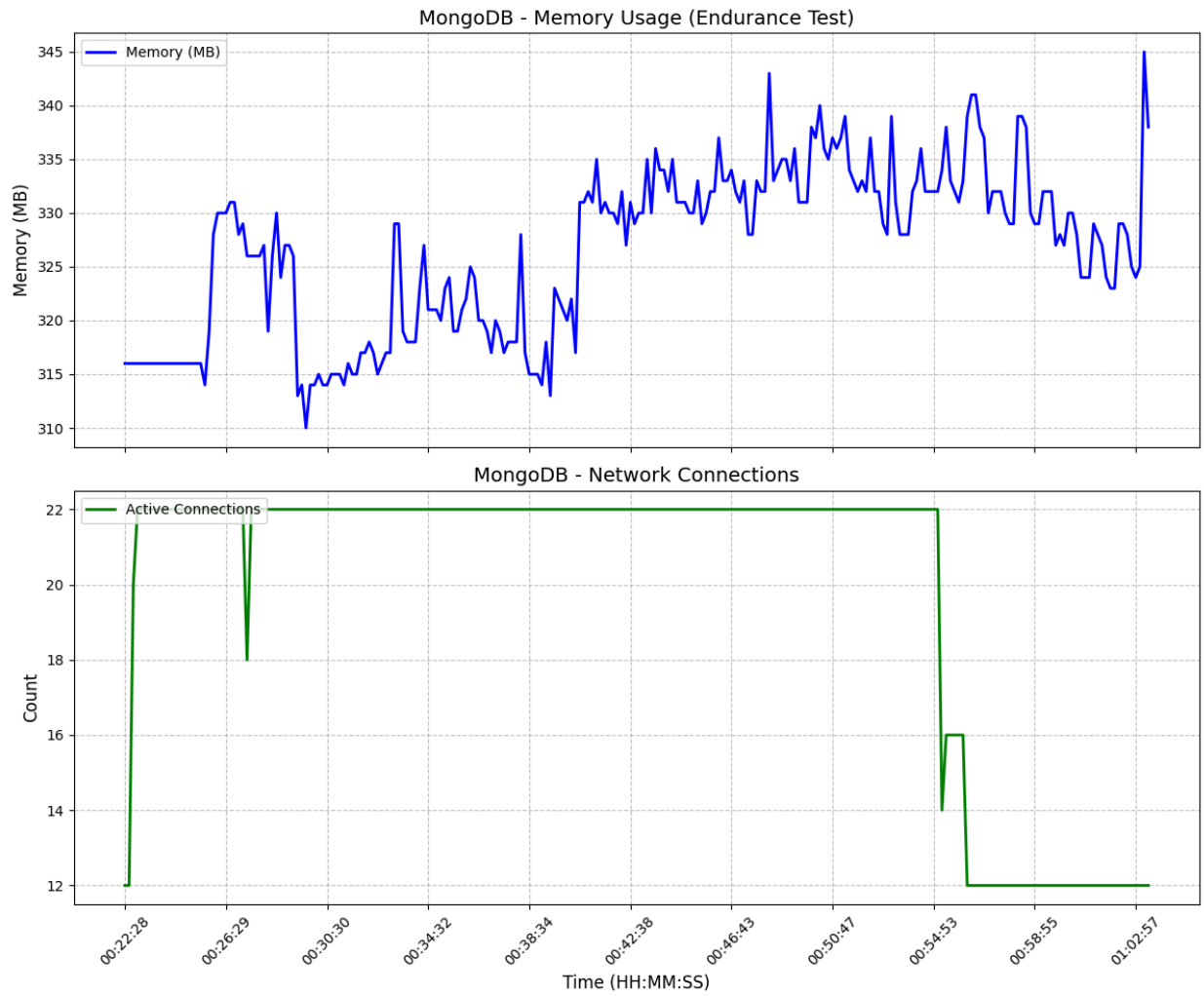
Database Metrics Snap

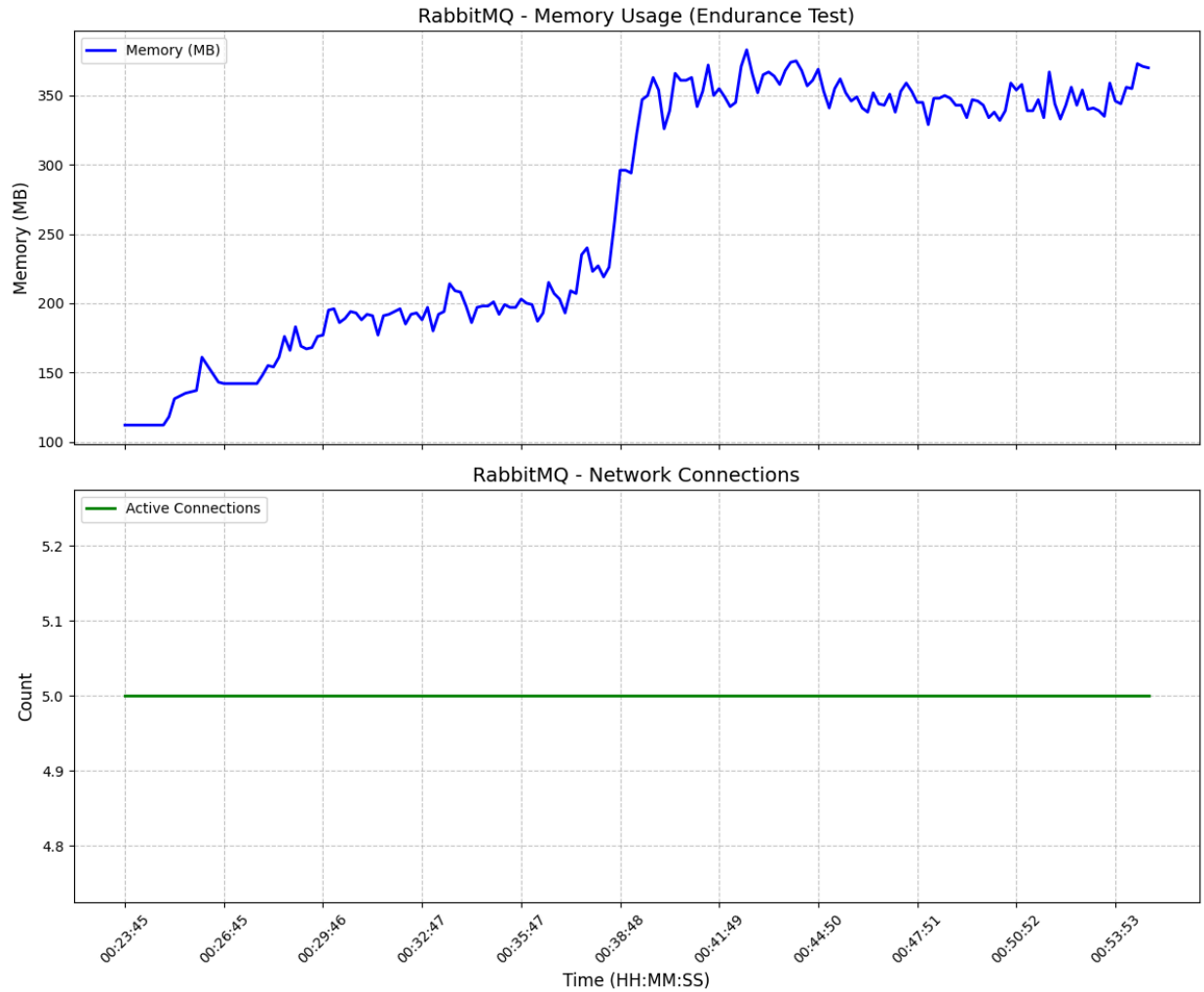
- MongoDB metrics in middle of the endurance test, which shows (QPS (Inserts) | Active Conns | Lock Wait (ms) | Cache Usage (MB) | Disk Write (MB))

12:39:22 AM	3092	17	2173647000	160.5	58.17
12:39:23 AM	2803	17	2174654000	155.7	50.97
12:39:24 AM	3337	17	2175686000	156.5	53.20
12:39:25 AM	3134	17	2176693000	156.0	50.79
12:39:26 AM	3421	17	2177700000	156.3	55.66
12:39:27 AM	3134	17	2178708000	155.9	56.88
12:39:28 AM	2388	17	2179721000	156.7	59.75
12:39:29 AM	2441	17	2180728000	158.5	41.09
12:39:30 AM	3081	17	2181740000	159.3	46.06
12:39:31 AM	1748	17	2182748000	154.6	27.32
12:39:32 AM	1812	17	2183758000	156.6	31.66
12:39:33 AM	3121	17	2184766000	154.9	52.78
12:39:34 AM	3264	17	2185785000	162.2	60.81
12:39:35 AM	2558	17	2186793000	157.5	45.54

Memory and Disk metrics of Consumer, RabbitMQ and MongoDB

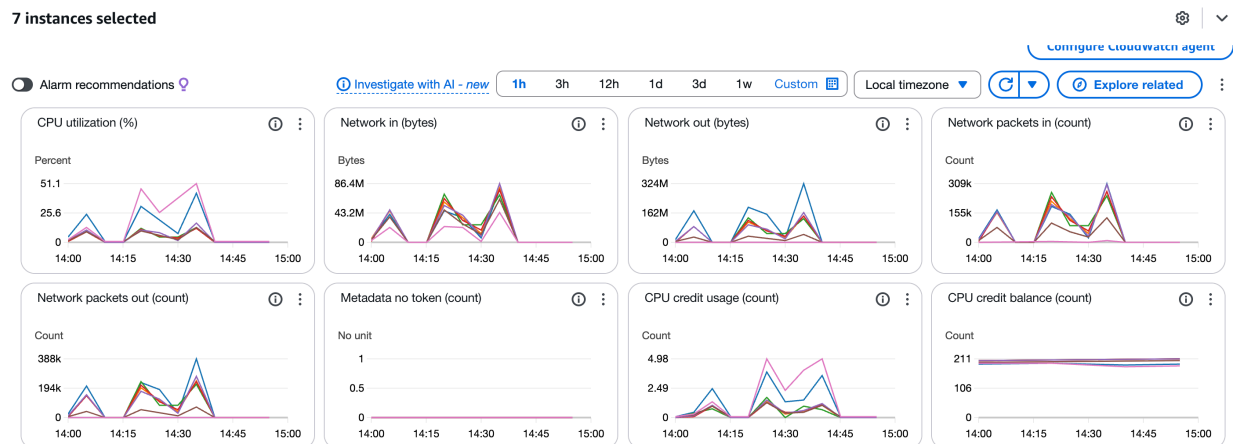




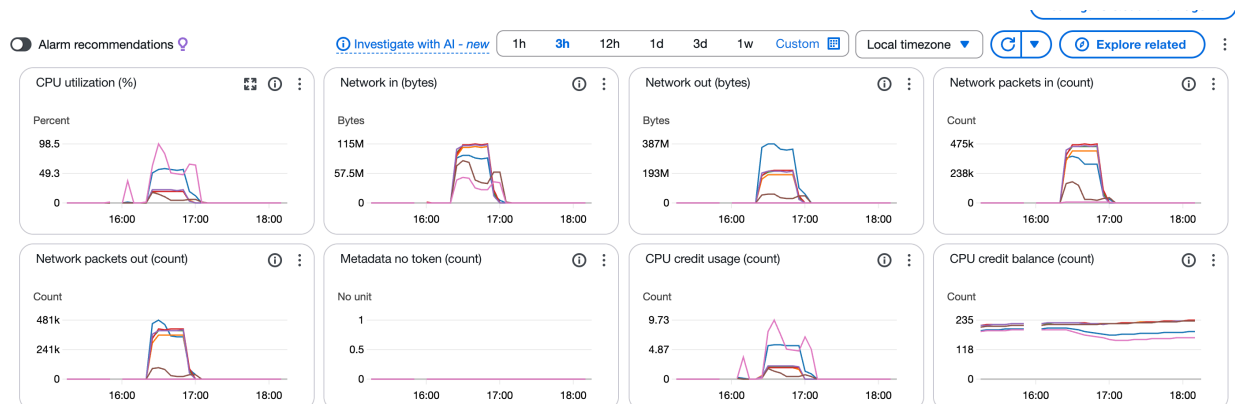


Instances Performance analysis on Baseline and Stress tests

7 instances selected



Instances Performance analysis on Endurance test



4. Configuration Details

Consumer Configuration:

```
public static final int BATCH_SIZE = 500;
public static final int FLUSH_INTERVAL_MS = 500;
public static final int WRITER_THREADS = 10;
public static final int BUFFER_SIZE = 10000;
public static final int MONGO_POOL_SIZE = 15;
```

Client Configuration:

```
public static final int NUM_THREADS = 256;
public static final int NUM_ROOMS = 20;
```

Conclusion

The chat system demonstrates a robust and scalable architecture capable of handling high-throughput workloads. This design successfully decouples real-time communication from data persistence, ensuring low latency for users even under heavy load.